Quiz Bowl RL Buzzer

Multiple-Choice Strategic Buzzing Under Incremental Clues

CS234 Final Project

Kathleen Weng
Imran Hassan
Ankit Aggarwal

March 2026

Problem

- Quiz bowl questions reveal evidence incrementally
- A good system must decide WHEN to buzz, not just WHAT to pick
- Buzz too early: higher risk of a wrong answer
- Buzz too late: lower strategic value, less chance to beat opponent
- We study this in a multiple-choice setting so the answer space is controlled and evaluation is reproducible

Background And Setup

Sequential decision problem

- Model quiz bowl as sequential decision over partial clues
- Question prefixes over time
- K=4 answer options
- One correct option + three distractors

Two policy families

Belief-feature buzzers

- Threshold, softmax-profile, Bayesian
- PPO on structured observations

T5 text-policy buzzers

- End-to-end supervised + PPO

Why Multiple Choice?

Advantages

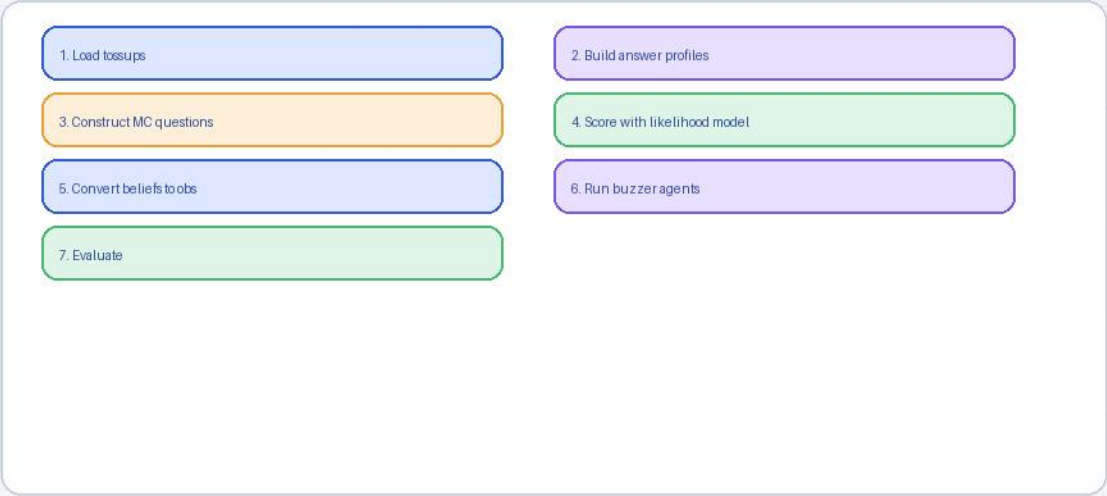
- Eliminates aliasing & grading complexity
- Isolates the buzzing decision itself
- Controlled, reproducible evaluation

Challenge

- Naive distractors create artifacts
- Answer generation quality matters

Design goals

- Keep the answer space constrained
- Make options hard enough that the agent must use clues
- Anti-artifact guards: alias collision, token overlap, length ratio, question-text overlap checks


1. Load tossups
2. Build answer profiles
3. Construct MC questions
4. Score with likelihood model
5. Convert beliefs to obs
6. Run buzzer agents
7. Evaluate

We study pyramidal quiz bowl under a restricted MC action space + PPO.

- Multiple-choice POMDP formulation for quiz bowl
- PPO to learn when to wait vs. buzz and answer
- Combination not explored in prior work
- Directly evaluate whether learned policies rely on clues or answer-choice patterns
- Anti-artifact MC construction with multiple distractor strategies
- Calibration-focused evaluation (ECE, Brier, S_q) beyond raw accuracy

Metric	Supervised	PPO (50 iter)	Direction
Overall accuracy	68%	64%	explore
Early buzz acc (clues 1-2)	41%	▨	▨
Late buzz acc (clues 5-6)	79%	▨	▨
Average reward	+0.31	+0.42	up 35%
Mean buzz position	2.3 clues	3.1 clues	more cautious
ECE	0.18	0.15	better cal.

PPO trades accuracy for more cautious buzzing; calibration improves.

Why Is PPO Accuracy Lower Than Supervised?

Not a bug — it's the intended trade-off

- Supervised model always answers at the end (forced guess)
- PPO learns to WAIT when uncertain
- Waiting avoids low-confidence wrong guesses
- But some waited questions time out unanswered
- Result: fewer attempts, so raw accuracy dips

68% supervised vs 64% PPO on overall accuracy

What PPO gains instead

- Average reward up 35% (+0.31 to +0.42)
- More cautious buzzing (2.3 to 3.1 clues)
- Better calibration (ECE 0.18 to 0.15)
- Fewer confident-but-wrong answers
- Agent delays on hard questions, commits on easy ones

PPO optimizes expected reward, not raw accuracy.
Lower accuracy + better timing = higher overall value.

Why PPO accuracy is lower

Quiz bowl MC stopping = optimal stopping with improving evidence

Pyramidal tossup

A question is read clue-by-clue. Early clues are obscure; later clues are easier. That means the state improves over time.

- obscure clue only experts get
- stronger clue narrows the field
- giveaway clue makes the answer obvious

RL intuition
The answer space is finite; the hard part is deciding when to stop listening.

Multiple-choice twist

Options are fixed from the start:

A A: option A	B B: option B
C C: option C	D D: option D

RL question
At each prefix, should the policy WAIT or BUZZ with one of the answers?

MC stopping intuition

Section 1: posterior sharpens as cluesarrive

next: see the same question as an RL stopping problem

As clues arrive, posterior mass concentrates on one option

prefix t, only the clue seen so far is visible

only a very obscure clue is visible

WAIT

Not enough value yet: waiting still has positive future benefit because one more clue can move the posterior.

In a finite answer space, the policy does not discover new options. It decides whether the current posterior is sharp enough to commit.

posterior sharpening

As clues arrive, posterior mass concentrates on one option

prefix t, only the clue seen so far is visible

a second clue rules out one distractor

WAIT

Not enough value yet: waiting still has positive future benefit because one more clue can move the posterior.

In a finite answer space, the policy does not discover new options. It decides whether the current posterior is sharp enough to commit.

posterior sharpening

As clues arrive, posterior mass concentrates on one option

prefix t, only the clue seen so far is visible

a third clue makes option A increasingly plausible

WAIT

Not enough value yet: waiting still has positive future benefit because one more clue can move the posterior.

In a finite answer space, the policy does not discover new options. It decides whether the current posterior is sharp enough to commit.

posterior sharpening

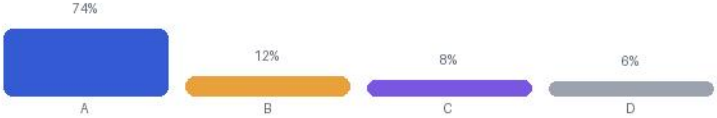
As clues arrive, posterior mass concentrates on one option

prefix t, only the clue seen so far is visible

a later clue makes A strongly favored

WAIT

Not enough value yet: waiting still has positive future benefit because one more clue can move the posterior.



In a finite answer space, the policy does not discover new options. It decides whether the current posterior is sharp enough to commit.

posterior sharpening

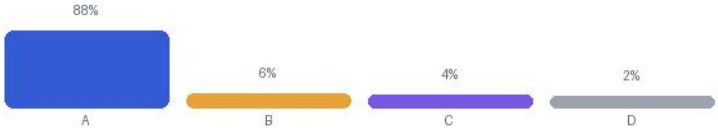
As clues arrive, posterior mass concentrates on one option

prefix t, only the clue seen so far is visible

one more clue pushes the argmax over the commit threshold

BUZZ

Now the best argmax is good enough: expected commit value exceeds the value of waiting.



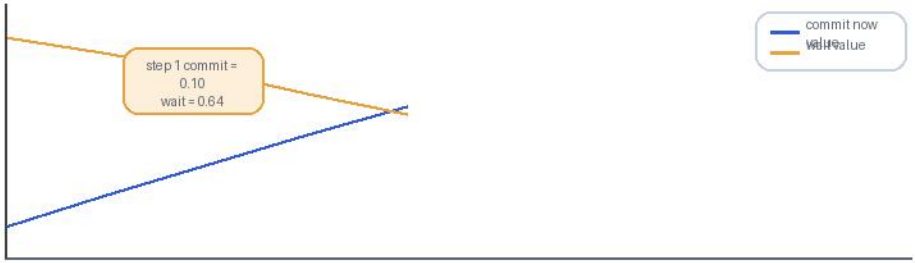
In a finite answer space, the policy does not discover new options. It decides whether the current posterior is sharp enough to commit.

posterior sharpening

Section 2: stop when commit value beats waiting

next: see the same question as an RL stopping problem

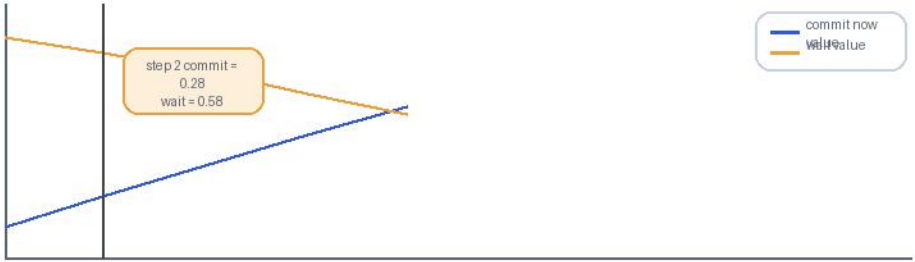
Buzz when committing beats the continuation value of waiting



Early on, future clues are worth a lot, so WAIT dominates.

commit vs wait

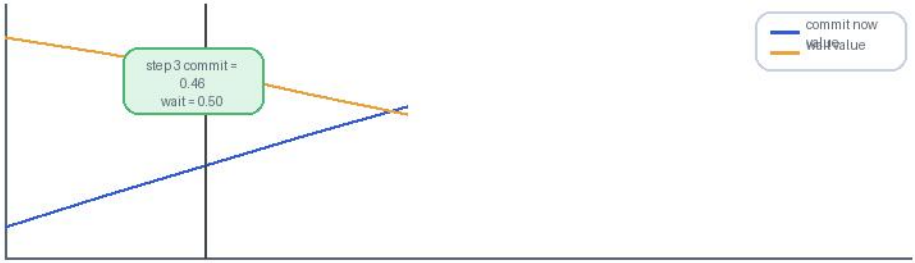
Buzz when committing beats the continuation value of waiting



The curves get closer as evidence improves.

commit vs wait

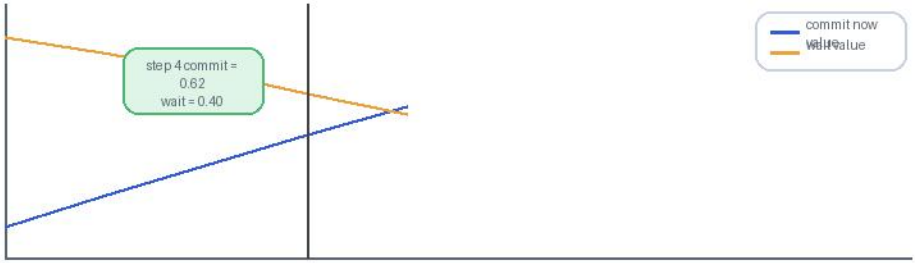
Buzz when committing beats the continuation value of waiting



Near the crossing, timing is the whole problem.

commit vs wait

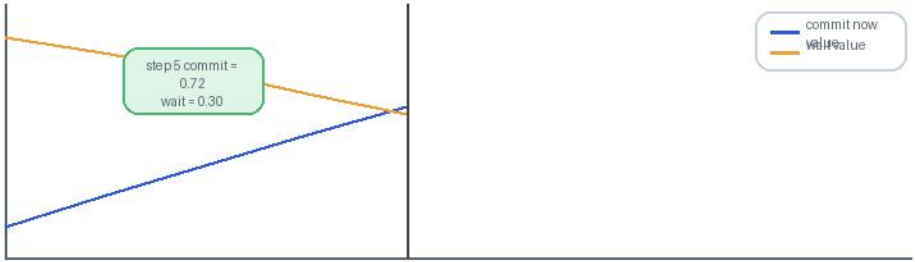
Buzz when committing beats the continuation value of waiting



Once the current curve crosses the continuation curve, BUZZ becomes optimal.

commit vs wait

Buzz when committing beats the continuation value of waiting



Later, waiting only delays an answer that is already good enough.

commit vs wait

Section 3: abstain means continue, not NOTA

next: see the same question as an RL stopping problem

Important semantic point: ABSTAIN is not 'none of the above'

The gold answer is already in the option set

A
A: option A

B
B: option B

C
C: option C

D
D: option D

Abstaining means the full value of one more clue is higher than the value of committing now.

What changes over time

ABSTAIN

means continuation because another clue is worth it

NOTA

Only architecture or loss design can make abstain behave like a reject option sink. That is an optimization artifact, not the Bayes-optimal semantics.

Important semantic point: ABSTAIN is not 'none of the above'

The gold answer is already in the option set

A
A: option A

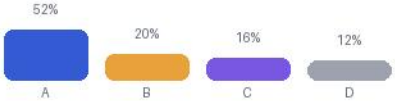
B
B: option B

C
C: option C

D
D: option D

Abstaining means the full value of one more clue is higher than the value of committing now.

What changes over time



ABSTAIN

means continuation because another clue is worth it

NOTA

The gold answer can already be present while waiting is still rational because another clue has positive value of information.

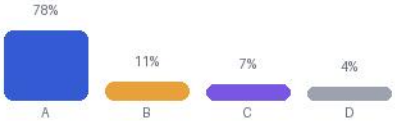
Important semantic point: ABSTAIN is not 'none of the above'

The gold answer is already in the option set

- A: option A
- B: option B
- C: option C
- D: option D

Abstaining means the full value of one more clue is higher than the value of committing now.

What changes over time



ABSTAIN

means continuation because another clue is worth it

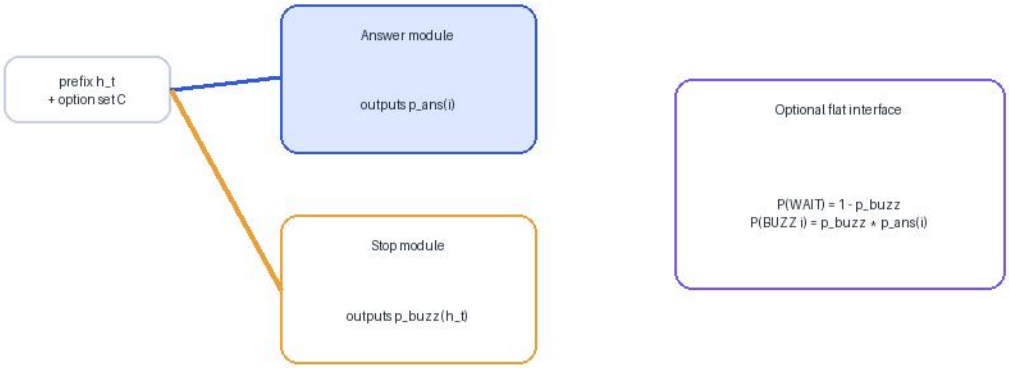
NOTA

So abstain is about continuation value, not exhaustivity of the answer set.

Section 4: factor answer quality and stop timing

next: see the same question as an RL stopping problem

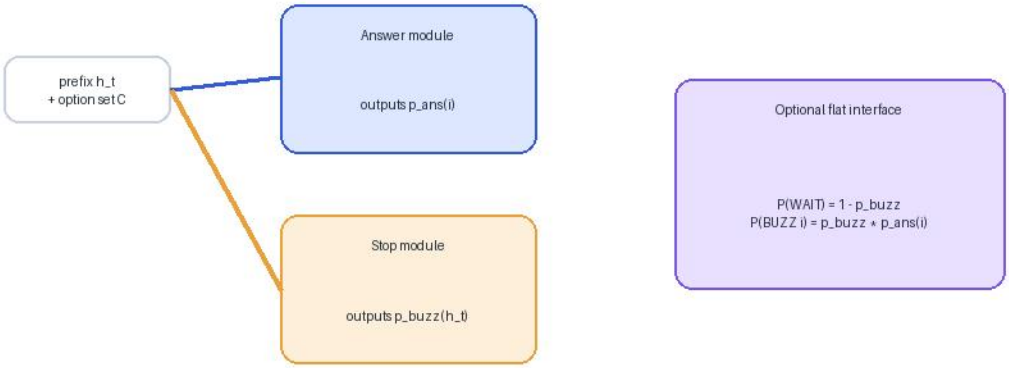
Implementation intuition: separate answer quality from stop timing



Same decision semantics, cleaner implementation: factor stop and answer internally, then expose a flat API only if the RL stack needs it.

factor stop and answer

Implementation intuition: separate answer quality from stop timing



Same decision semantics, cleaner implementation: factor stop and answer internally, then expose a flat API only if the RL stack needs it.

factor stop and answer

Section 5: one clean mental model

next: see the same question as an RL stopping problem

A practical training recipe

1. answer model first

learn p_ans(i | h_t)

2. hazard-loss warm start

learn when to commit

3. PPO fine-tuning

optimize task reward

RL then learns mostly the stopping problem, not which answer plus whether to stop all at once.

Training setup in code

scripts/build_mc_dataset.py --smoke -> scripts/run_baselines.py --smoke -> scripts/train_ppo.py --smoke

Data + baseline prep

- build MC prefixes
- set distractor strategy
- run baseline sweeps

RL fine-tune

- initialize policy
- optimize timing reward
- trade speed vs accuracy

Artifacts

artifacts/smoke/
- mc_dataset.json
- baseline_summary.json
- ppo_runs.json

Evaluation protocol

Core metrics

- mean S_q
- buzz accuracy
- mean buzz step
- ECE and Brier

Control checks

- choices-only baseline
- shuffle test
- alias substitution
- per-category slices

Evaluation source of truth: artifacts/smoke/evaluation_report.json

Smoke results snapshot

Best baseline (mean S_q)

always_final

- mean S_q = 0.386
- buzz acc = 38.6%
- mean step = 4.05
- n = 44

PPO smoke

- mean S_q = 0.326
- buzz acc = 34.1%
- reward-like = -0.476
- ECE/Brier = 0.099/0.013

Full eval: S_q=0.243, accuracy=38.6%, mean step=3.50

Quiz bowl MC stopping, in one picture

1. clues arrive from hard to easy

2. posterior over fixed answer choices sharpens

Good mental model: selective prediction + optimal stopping + fixed answer set

For RL people

At each prefix, compare the value of committing now to the value of one more clue.

summary

Quiz bowl MC stopping, in one picture

1. clues arrive from hard to easy

2. posterior over fixed answer choices sharpens

3. the RL decision is mostly when to stop listening

Good mental model: selective prediction + optimal stopping + fixed answer set

For RL people

At each prefix, compare the value of committing now to the value of one more clue.

If you keep a flat action interface, factor it internally:
P(WAIT)=1-p_buzz
P(BUZZ i)=p_buzz * p_ans(i)

Quiz bowl MC stopping, in one picture

1. clues arrive from hard to easy
2. posterior over fixed answer choices sharpens
3. the RL decision is mostly when to stop listening
4. abstain means another clue is worth it, not 'none of these are right'

Good mental model: selective prediction + optimal stopping + fixed answer set.

For RL people

At each prefix, compare the value of committing now to the value of one more clue.

If you keep a flat action interface, factor it internally:
P(WAIT)=1-p_buzz
P(BUZZ i)=p_buzz * p_ans(i)

Key takeaway: pyramidal quiz bowl turns MCQA into a stopping problem because information improves before action.

Metrics

- S_q (QANTA system score)
- Buzz accuracy
- Mean buzz step
- Calibration-at-buzz / ECE / Brier
- Per-category accuracy

Controls

- Choices-only baseline (no question text)
- Shuffle clue order
- Alias substitution
- Distractor difficulty variation

A buzzer can look strong while exploiting answer artifacts instead of clue content.

- Rodriguez et al. (2019) ⊡ Quizbowl: The case for incremental QA
- Schulman et al. ⊡ Proximal Policy Optimization (PPO)
- Raffel et al. ⊡ T5: Exploring the Limits of Transfer Learning
- Boyd-Graber & Daume (2013) ⊡ Bayesian thinking on your feet
- Boyd-Graber & Borschinger (2020) ⊡ What QA can learn from trivia nerds
- Balepur et al. (2025) ⊡ Test-time reasoners are strategic MC test-takers
- Kalai et al. (2025) ⊡ Why language models hallucinate
- UMD / QANTA (2024) ⊡ S_q evaluation metric